

Transformers and Attention Mechanisms: A Comprehensive Guide for Time-Series Classification

Part I: Foundational Principles of Attention

Chapter 1: The Imperative for a New Architecture: Limitations of Recurrent Models

To fully appreciate the paradigm shift introduced by the Transformer architecture, it is essential to first understand the fundamental limitations of the models that preceded it. For years, Recurrent Neural Networks (RNNs), and their more sophisticated variants like Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs)², were the state-of-the-art for processing sequential data. These models were instrumental in advancing fields like natural language processing and time-series analysis.³ However, their core architectural principles presented inherent constraints that ultimately hindered progress, especially as sequence lengths and model complexity grew. These limitations are not merely implementation details but are fundamental to the recurrent nature of these models, creating a clear and compelling need for a new approach.

1.1 The Sequential Computation Bottleneck

The defining characteristic of any recurrent model is its sequential processing of data. An RNN processes a sequence one element (or token) at a time, maintaining a "hidden state" that captures information from all previous elements. The hidden state at a given timestep t , denoted as h_t , is a function of the input at that timestep, x_t , and the hidden state from the previous timestep, h_{t-1} . This can be represented by the

recurrence relation $h_t = f(W h_{t-1} + U x_t)$, where f is a non-linear activation function and W and U are weight matrices.

This step-by-step computation creates a fundamental architectural bottleneck.⁴ The calculation for each timestep is dependent on the completion of the previous one. This inherent sequentiality severely limits the potential for parallelization across the time dimension of a sequence.³ While it is possible to process multiple independent sequences in parallel within a batch (a form of data parallelism), the computation

within a single long sequence remains stubbornly serial.⁶ This architectural constraint is fundamentally misaligned with the capabilities of modern hardware accelerators like Graphics Processing Units (GPUs) and Tensor Processing Units (TPUs), which are designed to perform thousands of similar operations simultaneously. As models grew larger and datasets expanded, this sequential bottleneck made training and inference progressively slower and more computationally expensive, creating a powerful incentive for an architecture that could process all elements of a sequence in parallel.⁵

1.2 The Challenge of Long-Range Dependencies

A primary goal of sequence modeling is to capture dependencies between elements, regardless of their distance from one another in the sequence. For example, in classifying a hand grip strategy, the initial muscle activation pattern at the beginning of the time-series is critically related to the stabilization phase at the end. RNNs, in theory, can capture these arbitrary long-term dependencies. In practice, however, they face significant mathematical hurdles known as the vanishing and exploding gradient problems.²

These problems arise during the training process, which typically uses an algorithm called Backpropagation Through Time (BPTT). During BPTT, the gradient of the loss function is propagated backward through the unrolled network, from the final timestep to the first. To update the weights associated with an early timestep t , the gradient must be repeatedly multiplied by the recurrent weight matrix W for each step it travels back in time. The gradient at an early timestep t can be approximated as $\partial \theta_t \partial L \approx 0$ if the gradients become too small.⁴ If the values in

W are small (less than 1), this repeated multiplication causes the gradient to shrink exponentially, eventually "vanishing" to near zero. Consequently, the model receives a

negligible signal to update its weights based on early inputs, making it impossible to learn long-range dependencies.¹ Conversely, if the values in

W are large (greater than 1), the gradient can grow exponentially and "explode," leading to unstable training and wildly fluctuating weight updates.⁵

More advanced architectures like LSTMs and GRUs were developed specifically to mitigate these issues.² They introduce sophisticated "gating mechanisms"—such as the forget gate, input gate, and output gate in LSTMs—that act as controllers of information flow.⁴ These gates can learn to selectively retain or discard information from the hidden state, effectively creating "shortcuts" that allow the gradient to flow more easily across many timesteps without vanishing. While these gating mechanisms were a significant improvement and made LSTMs and GRUs the standard for many years, they do not fundamentally solve the problem. They are a mitigation, not a cure. The path length for information to travel between distant tokens is still proportional to the distance between them, and for extremely long sequences, even these models struggle to maintain context and are susceptible to forgetting information from early in the sequence.¹ A new architectural paradigm was needed, one that could reduce the path length between any two tokens to a constant, thereby making the learning of long-range dependencies a more tractable problem.

1.3 The Information Bottleneck

In the context of sequence-to-sequence (Seq2Seq) tasks, such as machine translation or time-series forecasting, traditional models based on RNNs or LSTMs suffer from a critical design flaw known as the information bottleneck.⁷ These models typically employ an encoder-decoder structure. The encoder RNN processes the entire input sequence and compresses all of its information into a single, fixed-length vector, often referred to as the "context vector," which is simply the final hidden state of the encoder.⁸

This context vector is then passed to the decoder RNN, which must use it as the sole source of information to generate the entire output sequence. It is highly challenging, if not impossible, for a single vector of fixed dimensions to perfectly encapsulate all the nuances of a long and complex input sequence. As the input sequence gets longer, the model is forced to compress more information into the same fixed-size vector, inevitably leading to information loss.⁸ This design disproportionately affects

the beginning of the sequence, as the model's memory tends to be biased towards more recent inputs.⁵ This bottleneck was a major limitation that the first attention mechanisms were designed to address, by allowing the decoder to look back at

all the encoder's hidden states at each step, rather than relying on just the final one.¹⁰

The collective weight of these limitations—the sequential computation bottleneck preventing parallelization, the mathematical difficulty of learning long-range dependencies due to vanishing gradients, and the information bottleneck in encoder-decoder structures—created a clear and urgent need for a new architectural approach. The stage was set for the Transformer, a model that would dispense with recurrence entirely and rely on a different mechanism to relate sequence elements: attention.

Chapter 2: The Attention Mechanism: Intuition and Mathematical Formulation

The attention mechanism is the central innovation that powers the Transformer architecture. It was first introduced to address the information bottleneck in RNN-based encoder-decoder models but was later generalized into a powerful, standalone mechanism capable of relating all positions within a single sequence.¹⁰ Before delving into the full Transformer architecture, it is crucial to build a solid understanding of attention, both intuitively and mathematically. At its core, attention is a simple yet powerful concept that allows a model to dynamically weigh the importance of different parts of the input data when making a prediction, mimicking the human cognitive ability to focus on salient details.⁸

2.1 Intuition: Attention as a Differentiable Key-Value Lookup

Perhaps the most intuitive way to understand the attention mechanism is through the analogy of a modern information retrieval system, like a search engine or a database lookup. In this analogy, the input sequence is treated as a database of information. This database consists of a set of **Values** (V), which represent the actual content or features of each element in the sequence. Each Value is indexed by a corresponding **Key** (K), which acts as a descriptor or address for that piece of information.⁸

When the model needs to compute an updated representation for a specific element in the sequence, that element formulates a **Query (Q)**. This Query represents a question: "What information from the rest of the sequence is relevant to me?".⁸ The attention mechanism then performs the following steps⁸:

1. **Score Calculation:** The Query vector is compared against every Key vector in the database. This comparison, typically a dot product, produces a raw similarity score for each key-query pair. A high score indicates high relevance or alignment between the query and the key.
2. **Weight Normalization:** These raw scores, which can be any real number, are then passed through a softmax function. The softmax function converts the scores into a set of positive "attention weights" that sum to 1. These weights can be interpreted as a probability distribution, indicating how much "attention" the model should pay to each corresponding Value.
3. **Weighted Sum (Output):** Finally, the output for the given Query is calculated as a weighted sum of all the Value vectors in the database. The weights used in this sum are the attention weights calculated in the previous step.

This process is effectively a "soft" or "proportional" retrieval of information. Instead of retrieving a single, hard-coded value (like a standard dictionary lookup), the attention mechanism retrieves a blended combination of all values, with the blend being heavily biased towards the values whose keys were most relevant to the query. This entire process is composed of differentiable operations (matrix multiplication and softmax), which means it can be seamlessly integrated into a deep learning model and trained via gradient descent.

2.2 The Mathematics of Scaled Dot-Product Attention

The specific implementation of the attention mechanism used in the original Transformer paper is called **Scaled Dot-Product Attention**. Its operation is elegantly captured by a single, concise formula¹¹:

$$\text{Attention}(Q,K,V)=\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Let's dissect this formula step-by-step to understand its mathematical and practical significance. Here, Q (Queries), K (Keys), and V (Values) are matrices that pack together the query, key, and value vectors for all tokens in the sequence.

1. **Dot Product as Similarity (QKT):** The first operation is the matrix multiplication

of the Query matrix Q with the transpose of the Key matrix K^T . If Q has dimensions (N, d_k) and K has dimensions (M, d_k) , where N and M are sequence lengths and d_k is the dimension of the key/query vectors, the resulting matrix QK^T will have dimensions (N, M) . Each element (i, j) of this matrix is the dot product of the i -th query vector and the j -th key vector, representing their similarity or alignment. This single matrix multiplication efficiently computes the similarity score for every query with every key in parallel.

2. **The Scaling Factor ($1/d_k$):** After the dot product, the scores are scaled by dividing by d_k , the square root of the dimension of the key vectors.¹² This scaling factor is a critical detail for stabilizing the training process. For large values of d_k , the dot products can grow very large in magnitude. When these large values are fed into the softmax function, they can get pushed into regions where the function's gradient is extremely small (close to zero). This is the vanishing gradient problem, which can stall the learning process.¹¹ By scaling down the dot products, the arguments to the softmax are kept in a more reasonable range, ensuring that gradients remain stable and the model can train effectively.
3. **Softmax for Normalization:** The softmax function is applied along the last dimension of the scaled score matrix. This converts the raw alignment scores into a probability distribution of attention weights. Each row of the resulting attention matrix now contains values between 0 and 1 that sum to 1. These values dictate, for a given query, the proportion of attention to be paid to each key.
4. **Weighted Sum of Values (...V):** The final step is to multiply the attention weights matrix (the output of the softmax) by the Value matrix V . This computes the weighted sum of the value vectors. The intuition is that the values of tokens we want to focus on (those with high attention weights) are preserved, while irrelevant tokens (with weights near zero) are effectively "drowned out".⁸ The resulting matrix is the output of the attention layer, a new set of vectors where each vector is a contextualized representation of an input token, enriched with information from other relevant tokens in the sequence.

2.3 Practical PyTorch Example: Scaled Dot-Product Attention

The mathematical formula can be directly translated into a Python function using PyTorch, which highlights the elegance and efficiency of these matrix operations.

Python

```
import torch
import torch.nn as nn
import math
```

```
def scaled_dot_product_attention(q, k, v, mask=None):
```

```
    """
```

```
    Calculates the scaled dot-product attention.
```

```
    Args:
```

```
    q (torch.Tensor): Query tensor; shape (batch_size, num_heads, seq_len_q, d_k)
```

```
    k (torch.Tensor): Key tensor; shape (batch_size, num_heads, seq_len_k, d_k)
```

```
    v (torch.Tensor): Value tensor; shape (batch_size, num_heads, seq_len_v, d_v)
```

```
    mask (torch.Tensor, optional): Mask tensor to prevent attention to certain positions. Defaults to None.
```

```
    Returns:
```

```
    torch.Tensor: The output of the attention mechanism.
```

```
    torch.Tensor: The attention weights.
```

```
    """
```

```
    d_k = k.size(-1) # Get the dimension of the keys
```

```
    # Matmul for similarity scores: (batch, heads, seq_len_q, d_k) @ (batch, heads, d_k, seq_len_k) ->
    (batch, heads, seq_len_q, seq_len_k)
```

```
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
```

```
    # Apply mask if provided (e.g., for padding or causal attention)
```

```
    if mask is not None:
```

```
        scores = scores.masked_fill(mask == 0, -1e9) # Fill with a very small number where mask
    is 0
```

```
    # Apply softmax to get attention weights
```

```
    attention_weights = torch.softmax(scores, dim=-1)
```

```
    # Matmul with values to get the final output
```

```
    # (batch, heads, seq_len_q, seq_len_k) @ (batch, heads, seq_len_v, d_v) -> (batch, heads, seq_len_q,
    d_v)
```

```
    # Note: seq_len_k must be equal to seq_len_v
```

```
    output = torch.matmul(attention_weights, v)
```

```
return output, attention_weights
```

This function encapsulates the core logic of Scaled Dot-Product Attention. The true power of the Transformer architecture comes from how these Q, K, and V matrices are generated and how this fundamental attention operation is composed into larger blocks, which will be explored in the next chapter.

A crucial realization is that this attention mechanism facilitates a form of dynamic, content-based connectivity. Unlike a Convolutional Neural Network (CNN) with a fixed local receptive field or an RNN with a fixed connection to its immediate predecessor, attention allows the network to learn the connection strengths between any two elements in a sequence on the fly. The attention weight between two tokens is not predetermined by their position but is calculated based on their vector representations (their Q and K vectors). This means the network can learn to create strong, direct connections between any two tokens, regardless of their distance, if their content is relevant to the task at hand.¹⁰ For a task like hand grip classification, this dynamic connectivity is immensely powerful. It could allow the model to learn to directly link the initial phase of muscle activation with the final stabilization phase of the grip, even if these events are separated by hundreds of timesteps in the sensor data—a feat that is exceptionally difficult for traditional recurrent models.

Furthermore, the source of the Q, K, and V matrices defines the type of attention being performed, a distinction that is fundamental to the various Transformer architectures. When Q, K, and V are all derived from the *same* input sequence, it is called **self-attention**. Here, the sequence attends to itself to build richer, more contextualized representations of its own elements.¹² This is the primary mechanism within the Transformer's encoder. In contrast, when the Q vectors are derived from a target sequence (e.g., in a decoder) while the K and V vectors come from a source sequence (e.g., the output of an encoder), it is called

cross-attention. This is the mechanism that allows a decoder to focus on relevant parts of the input sequence while generating an output, and it is the cornerstone of encoder-decoder models for tasks like machine translation.¹² Understanding this distinction is key to navigating the taxonomy of Transformer models discussed in later chapters.

Part II: Deconstructing the Transformer: An Architectural Deep Dive

Chapter 3: The Canonical "Attention Is All You Need" Architecture

The 2017 paper "Attention Is All You Need" by Vaswani et al. introduced the Transformer, a model architecture that eschewed recurrence and convolutions entirely, relying solely on attention mechanisms to process sequential data. This chapter provides a detailed, component-by-component breakdown of this seminal architecture, explaining the function of each part and how they work in synergy to achieve state-of-the-art performance. The model's power stems not from a single, monolithic innovation but from the elegant composition of several key ideas that collectively overcome the limitations of previous models.

3.1 The Encoder-Decoder Framework

At a high level, the original Transformer follows a standard encoder-decoder framework, a design common in sequence-to-sequence (Seq2Seq) tasks like machine translation.⁹ The architecture consists of two main parts:

1. **The Encoder:** Its role is to process the entire input sequence and transform it into a sequence of continuous, contextualized representations. In the original paper, the encoder is a stack of $N=6$ identical layers.⁹
2. **The Decoder:** Its role is to take the encoder's output and generate the output sequence one element at a time, in an autoregressive manner. The decoder is also a stack of $N=6$ identical layers.⁹

The critical difference from earlier RNN-based Seq2Seq models lies in how information is passed from the encoder to the decoder. While RNN models would compress the entire input into a single fixed-length context vector, creating an information bottleneck, the Transformer encoder passes its *entire* sequence of output vectors, Z , to each layer of the decoder.⁹ This allows the decoder, via the

cross-attention mechanism, to look at every part of the original input sequence at every step of the generation process, thereby completely resolving the information bottleneck problem.

3.2 Multi-Head Attention: Capturing Diverse Relationships

Instead of using a single attention function, the Transformer employs "Multi-Head Attention." This design was motivated by the idea that a single attention mechanism might be forced to average all the different types of relationships between tokens, potentially missing out on important nuances. By using multiple "heads," the model can jointly attend to information from different representation subspaces at different positions.⁹

The mechanism works as follows ¹²:

1. **Projection:** The input Query, Key, and Value vectors (each of dimension d_{model}) are not used directly. Instead, they are first linearly projected h times using different, learned weight matrices (W_iQ, W_iK, W_iV). This projects the data into h lower-dimensional subspaces, each with dimensions d_k , d_k , and d_v respectively, where $d_k=d_v=d_{\text{model}}/h$.
2. **Parallel Attention:** Scaled Dot-Product Attention is then applied in parallel to each of these h projected sets of queries, keys, and values, yielding h separate output vectors.
3. **Concatenation and Final Projection:** The h output vectors are concatenated and then projected back to the original d_{model} dimension with another learned linear transformation (W_O).

The complete formula is:

$$\text{MultiHead}(Q,K,V)=\text{Concat}(\text{head}_1,\dots,\text{head}_h)W_O$$

where

$$\text{head}_i=\text{Attention}(QW_iQ,KW_iK,VW_iV)$$

This structure allows different attention heads to specialize and learn to capture different types of relationships in the data. For instance, when analyzing time-series data from a hand grip, one head might learn to track the overall low-frequency trend of applied force, while another might specialize in detecting high-frequency muscle

tremors or oscillations. This parallel processing of different feature subspaces results in a much richer and more nuanced representation of the input sequence.

3.3 Positional Encoding: The Solution to Permutation Invariance

A core consequence of relying solely on self-attention is that the model becomes permutation-invariant—it has no inherent sense of the order or position of the tokens in a sequence. If the input embeddings were fed directly into the attention mechanism, the sentences "The dog bit the man" and "The man bit the dog" would produce identical representations, which is clearly undesirable.

To solve this, the Transformer injects information about the position of each token into the input embeddings. The original paper proposed a simple yet effective method using fixed sine and cosine functions of varying frequencies ¹⁴:

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}}) \quad PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

Here, pos is the position of the token in the sequence (from 0 to sequence length - 1), and i is the dimension index (from 0 to $d_{model}/2-1$). This creates a unique positional vector for each position in the sequence. This vector is then simply added to the corresponding token embedding.

The choice of sinusoidal functions is deliberate. Firstly, it provides a unique encoding for each position. Secondly, and more importantly, it allows the model to easily learn to attend to relative positions. This is because for any fixed offset k , the positional encoding at position $pos+k$ can be represented as a linear function of the encoding at position pos , a property that does not hold for simpler encoding schemes like learned positional embeddings. This fixed encoding scheme also has the advantage of being able to generalize to sequence lengths longer than those seen during training.

3.4 Residual Connections and Layer Normalization

The Transformer architecture is deep, composed of multiple stacked encoder and decoder layers. Training such deep networks is notoriously difficult due to the vanishing gradient problem. To address this, the Transformer heavily relies on two

standard techniques: residual connections and layer normalization.⁹

Each sub-layer within an encoder or decoder layer (i.e., the multi-head attention module and the feed-forward network) is wrapped in a residual connection, followed by a layer normalization step. The output of a sub-layer is thus computed as:

$\text{LayerNorm}(x + \text{Dropout}(\text{Sublayer}(x)))$

- **Residual Connections:** The "add" part of the operation, where the input to the sub-layer (x) is added to the output of the sub-layer ($\text{Sublayer}(x)$), creates a shortcut. This allows the gradient to flow more directly through the network during backpropagation, bypassing layers if necessary, which is crucial for enabling the training of very deep models.
- **Layer Normalization:** This technique stabilizes the training process by normalizing the inputs to each sub-layer. It computes the mean and variance of the activations for each individual training example across its feature dimension (d_{model}) and uses them to re-center and re-scale the activations.¹⁵ The formula for the mean (μ) and standard deviation (σ) for a layer's activations (a) with H hidden units is:

$$\mu = \frac{1}{H} \sum_{i=1}^H a_i \text{ and } \sigma = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i - \mu)^2}$$

Unlike Batch Normalization, which normalizes across the batch dimension, Layer Normalization is independent of the batch size and the statistics of other examples in the batch. This makes it particularly well-suited for sequence processing tasks where sequence lengths can vary. The placement of the normalization layer has been a subject of research; placing it *before* the sub-layer (Pre-LN) has been shown to lead to more stable training and can eliminate the need for learning rate warm-up schedules.

3.5 Position-Wise Feed-Forward Network (FFN)

In addition to the attention sub-layers, each encoder and decoder layer contains a fully connected, position-wise feed-forward network (FFN). This network consists of two linear transformations with a Rectified Linear Unit (ReLU) activation function in between¹⁶:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

This FFN is applied independently and identically to each position in the sequence. While the multi-head attention sub-layers are responsible for mixing information between different

positions in the sequence, the FFN's role is to provide a rich, non-linear transformation of the information at each position independently. It expands the representation to a higher dimension (typically $d_{ff}=2048$ when $d_{model}=512$) and then projects it back, allowing the model to learn more complex patterns and relationships within each token's representation. The combination of these components results in a powerful and efficient architecture. The following table, adapted from the original "Attention Is All You Need" paper, quantitatively summarizes the key advantages of the self-attention mechanism over its recurrent and convolutional counterparts.⁹

Layer Type	Computational Complexity per Layer	Sequential Operations	Maximum Path Length
Recurrent (RNN/LSTM)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional (kernel size k)	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(n/k)$
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$

Table 3.1: A quantitative comparison of different layer types for sequence processing, where n is the sequence length and d is the representation dimension. The self-attention layer connects all positions with a constant number of sequential operations and a constant maximum path length, enabling superior modeling of long-range dependencies and greater parallelization.

The true ingenuity of the Transformer is not just in the attention mechanism itself, but in the elegant engineering that combines these different modules. Each component addresses a potential weakness of another, creating a robust and synergistic whole. Self-attention solves the sequential bottleneck of RNNs but introduces permutation invariance. Positional encoding is then added specifically to solve this new problem by re-injecting order information. Stacking these layers deeply could lead to vanishing gradients, so residual connections are included to ensure stable gradient flow. The outputs of these layers could have inconsistent distributions, so layer normalization is applied to stabilize training. This intricate interplay reveals the Transformer as a masterclass in deep learning architecture design, where each part has a clear and necessary role in enabling the others to function effectively.

Chapter 4: Building a Transformer from First Principles in PyTorch

Translating the theoretical concepts of the Transformer architecture into functional code is a critical step toward practical application. This chapter provides a step-by-step guide to implementing the core components of the Transformer model from scratch using PyTorch. By building each module—from Positional Encoding to the final Encoder and Decoder stacks—we can demystify the architecture and create a reusable foundation for custom tasks like time-series classification. The modular nature of the Transformer lends itself well to an object-oriented implementation, where each conceptual block becomes a self-contained `nn.Module`.

4.1 Setting up the Environment

Before beginning, it is essential to have a working Python environment with PyTorch installed. The following libraries are required for this implementation:

- **PyTorch:** For building and training the neural network.
- **math:** For mathematical operations like `sqrt`.

These can be installed via package managers like `pip` or `conda`.

```
Python
```

```
# Essential imports for building the Transformer
```

```
import torch
```

```
import torch.nn as nn
```

```
import math
```

4.2 Implementing the Building Blocks

We will now implement each of the fundamental components of the Transformer as a distinct PyTorch class.

4.2.1 Positional Encoding

The PositionalEncoding module generates and adds the sinusoidal position vectors to the input embeddings to provide the model with information about the sequence order.

Python

```
# Code adapted from PyTorch tutorials and S153, S161
class PositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_seq_length: int = 5000, dropout: float = 0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        # Create a matrix for positional encodings of shape (max_seq_length, d_model)
        pe = torch.zeros(max_seq_length, d_model)

        # Create a tensor for position indices (0, 1, 2,..., max_seq_length-1)
        position = torch.arange(0, max_seq_length, dtype=torch.float).unsqueeze(1)

        # Calculate the division term for the sine and cosine functions
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) /
d_model))

        # Apply sine to even indices in the d_model dimension
        pe[:, 0::2] = torch.sin(position * div_term)

        # Apply cosine to odd indices in the d_model dimension
        pe[:, 1::2] = torch.cos(position * div_term)

        # Add a batch dimension and register 'pe' as a buffer.
        # Buffers are part of the model's state but are not considered parameters.
        self.register_buffer('pe', pe.unsqueeze(0))
```

```

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Adds positional encoding to the input tensor.
    Args:
        x: Input tensor of shape (batch_size, seq_length, d_model)
    Returns:
        Tensor with positional information added.
    """
    # Add positional encoding to the input tensor x
    # x.size(1) is the sequence length of the current batch
    x = x + self.pe[:, :x.size(1)]
    return self.dropout(x)

```

4.2.2 Multi-Head Attention

The MultiHeadAttention module implements the core attention mechanism, splitting the input into multiple heads to capture diverse relationships in parallel.

Python

```

# Code adapted from S153
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads # Dimension of each head

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

```



```

def scaled_dot_product_attention(self, Q, K, V, mask=None):
    attn_scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
    if mask is not None:
        attn_scores = attn_scores.masked_fill(mask == 0, -1e9)
    attn_probs = torch.softmax(attn_scores, dim=-1)
    output = torch.matmul(attn_probs, V)
    return output

def split_heads(self, x: torch.Tensor) -> torch.Tensor:
    batch_size, seq_length, d_model = x.size()
    return x.view(batch_size, seq_length, self.num_heads, self.d_k).transpose(1, 2)

def combine_heads(self, x: torch.Tensor) -> torch.Tensor:
    batch_size, _, seq_length, d_k = x.size()
    return x.transpose(1, 2).contiguous().view(batch_size, seq_length, self.d_model)

def forward(self, Q, K, V, mask=None):
    Q = self.split_heads(self.W_q(Q))
    K = self.split_heads(self.W_k(K))
    V = self.split_heads(self.W_v(V))

    attn_output = self.scaled_dot_product_attention(Q, K, V, mask)
    output = self.W_o(self.combine_heads(attn_output))
    return output

```

4.2.3 Position-Wise Feed-Forward Network

The PositionWiseFeedForward module applies a two-layer fully connected network to each position independently.

Python

```

# Code adapted from S153
class PositionWiseFeedForward(nn.Module):

```

```

def __init__(self, d_model: int, d_ff: int):
    super().__init__()
    self.fc1 = nn.Linear(d_model, d_ff)
    self.fc2 = nn.Linear(d_ff, d_model)
    self.relu = nn.ReLU()

def forward(self, x: torch.Tensor) -> torch.Tensor:
    return self.fc2(self.relu(self.fc1(x)))

```

4.3 Assembling the Transformer Layers

With the basic building blocks defined, we can now assemble them into the Encoder and Decoder layers.

4.3.1 Encoder Layer

The EncoderLayer encapsulates one layer of the encoder stack, containing a self-attention module and a feed-forward network, each with residual connections and layer normalization.

Python

```

# Code adapted from S153
class EncoderLayer(nn.Module):
    def __init__(self, d_model: int, num_heads: int, d_ff: int, dropout: float):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = PositionWiseFeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

```

```

def forward(self, x: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
    # Self-attention sub-layer
    attn_output = self.self_attn(x, x, x, mask)
    x = self.norm1(x + self.dropout(attn_output)) # Add & Norm

    # Feed-forward sub-layer
    ff_output = self.feed_forward(x)
    x = self.norm2(x + self.dropout(ff_output)) # Add & Norm

    return x

```

4.3.2 Decoder Layer

The DecoderLayer is similar but includes an additional cross-attention module to attend to the output of the encoder.

Python

```

# Code adapted from S153
class DecoderLayer(nn.Module):
    def __init__(self, d_model: int, num_heads: int, d_ff: int, dropout: float):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.cross_attn = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = PositionWiseFeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor, enc_output: torch.Tensor, src_mask: torch.Tensor, tgt_mask:
torch.Tensor) -> torch.Tensor:
        # Masked self-attention sub-layer

```

```

    attn_output = self.self_attn(x, x, x, tgt_mask)
    x = self.norm1(x + self.dropout(attn_output)) # Add & Norm

    # Cross-attention sub-layer
    cross_attn_output = self.cross_attn(x, enc_output, enc_output, src_mask)
    x = self.norm2(x + self.dropout(cross_attn_output)) # Add & Norm

    # Feed-forward sub-layer
    ff_output = self.feed_forward(x)
    x = self.norm3(x + self.dropout(ff_output)) # Add & Norm

    return x

```

4.4 Assembling the Full Transformer

Finally, the Transformer class combines all the components: embedding layers, positional encoding, stacks of encoder and decoder layers, and a final linear output layer.

Python

```

# Code adapted from S153, S154
class Transformer(nn.Module):
    def __init__(self, src_vocab_size: int, tgt_vocab_size: int, d_model: int, num_heads: int, num_layers:
int, d_ff: int, max_seq_length: int, dropout: float):
        super().__init__()
        self.encoder_embedding = nn.Embedding(src_vocab_size, d_model)
        self.decoder_embedding = nn.Embedding(tgt_vocab_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model, max_seq_length)

        self.encoder_layers = nn.ModuleList([EncoderLayer(d_model, num_heads, d_ff,
dropout) for _ in range(num_layers)])
        self.decoder_layers = nn.ModuleList()

```

```

self.fc = nn.Linear(d_model, tgt_vocab_size)
self.dropout = nn.Dropout(dropout)

def generate_mask(self, src, tgt):
    # Mask for padding tokens in the source sequence
    src_mask = (src!= 0).unsqueeze(1).unsqueeze(2)
    # Mask for padding tokens in the target sequence
    tgt_pad_mask = (tgt!= 0).unsqueeze(1).unsqueeze(2)
    seq_length = tgt.size(1)
    # Causal mask to prevent attention to future tokens in the target sequence
    nopeak_mask = (1 - torch.triu(torch.ones(1, seq_length, seq_length),
diagonal=1)).bool()
    # Combine padding mask and causal mask for the target
    tgt_mask = tgt_pad_mask & nopeak_mask
    return src_mask, tgt_mask

def forward(self, src: torch.Tensor, tgt: torch.Tensor) -> torch.Tensor:
    src_mask, tgt_mask = self.generate_mask(src, tgt)

    # Process source sequence through encoder
    src_embedded =
self.dropout(self.positional_encoding(self.encoder_embedding(src)))
    enc_output = src_embedded
    for enc_layer in self.encoder_layers:
        enc_output = enc_layer(enc_output, src_mask)

    # Process target sequence through decoder
    tgt_embedded =
self.dropout(self.positional_encoding(self.decoder_embedding(tgt)))
    dec_output = tgt_embedded
    for dec_layer in self.decoder_layers:
        dec_output = dec_layer(dec_output, enc_output, src_mask, tgt_mask)

    # Final linear layer to get logits
    output = self.fc(dec_output)
    return output

```

This from-scratch implementation demonstrates that the seemingly complex Transformer is, in fact, a composition of simpler, well-defined modules. This

modularity is a key reason for its adaptability and success, as it allows researchers to easily modify, replace, or analyze individual components. The following chapter will build upon this foundation to adapt the architecture for the specific task of time-series classification.

Chapter 5: Adapting Transformers for Hand Grip Strategy Classification

While the canonical Transformer was designed for natural language processing tasks like machine translation, its core ability to model dependencies in sequential data makes it a powerful tool for time-series analysis. This chapter provides a comprehensive, end-to-end guide on adapting the Transformer architecture for the specific task of classifying multivariate time-series data, such as that collected from sensors during hand grip motions. This involves reframing the problem, developing appropriate data preprocessing techniques for continuous signals, and designing a suitable model head for classification.

5.1 Time-Series as a Sequence Problem

The first step is to frame the classification of a hand grip strategy as a sequence classification problem. A single trial of a hand grip produces a multivariate time-series, which can be represented as a tensor of shape (C,T) , where C is the number of sensor channels (e.g., different EMG electrodes, force sensors) and T is the number of timesteps. For a Transformer, it is more convenient to work with the shape (T,C) . The goal is to input this entire sequence into a model and output a single class label (e.g., "power grip," "precision grip," "release").

For this type of task, where the goal is to *understand* the entire input sequence to produce a single output, the full encoder-decoder architecture is often unnecessary and computationally expensive. Instead, an **encoder-only architecture** is more suitable.¹⁷ The encoder's role is to process the input time-series and generate a rich, contextualized representation. This final representation can then be fed into a simple classification head to make the final prediction. This approach is analogous to how BERT is used for sentence classification and provides a more direct and efficient

solution for this problem.

5.2 Data Formatting and Preprocessing for Time-Series

Transformers were designed for discrete tokens (words), whereas time-series data is often continuous. This necessitates a "tokenization" strategy for the continuous signal, a critical design choice that fundamentally impacts what the model can learn.

5.2.1 Patching: The Tokenization of Continuous Data

A common and effective technique is **patching** or **windowing**.¹⁸ The input time-series of length

T is segmented into a sequence of smaller, fixed-size patches (or windows). For example, a time-series of 1000 timesteps could be divided into 100 non-overlapping patches of 10 timesteps each. Each of these patches is then treated as a single "token."

This approach has two main benefits:

1. **Sequence Length Reduction:** It reduces the length of the sequence fed into the Transformer, mitigating the $O(n^2)$ complexity of the self-attention mechanism.
2. **Local Feature Learning:** It allows the model to learn local patterns within each patch.

Each patch, which is a matrix of shape $(\text{patch_length}, \text{num_features})$, is typically flattened and then passed through a linear projection layer to create a patch embedding of dimension d_{model} . This results in a sequence of patch embeddings of shape $(\text{num_patches}, d_{\text{model}})$, which is the format expected by the Transformer encoder. The choice of patch size is a crucial hyperparameter: larger patches are more computationally efficient but may average out important high-frequency details, while smaller patches preserve local information at a higher computational cost.

5.2.2 Handling Multivariate Data

For multivariate time-series, such as data from multiple EMG sensors, there are two primary strategies for creating the patch embeddings ¹⁹:

1. **Channel-Mixing:** In this approach, at each timestep within a patch, the features from all C channels are treated as a single vector. The patch, of shape $(\text{patch_length}, C)$, is flattened into a vector of size $(\text{patch_length} \times C)$ and then linearly projected into the d_{model} embedding space. This method allows the model to learn correlations and interactions between different sensor channels from the very first layer.
2. **Channel-Independent:** Here, each of the C channels is treated as an independent sequence. Each channel is patched and embedded separately, resulting in C different sequences of patch embeddings. These sequences can be processed by separate Transformer encoders, or more commonly, concatenated along the sequence dimension and processed by a single encoder. This approach may be useful if the channels represent very different modalities.

For most sensor fusion tasks like hand grip analysis, the channel-mixing approach is often preferred as it encourages the model to learn the synergistic patterns across different muscles and sensors.

5.3 Designing the Classification Head

After the Transformer encoder processes the sequence of patch embeddings, it outputs a sequence of contextualized embeddings of the same shape, $(\text{batch_size}, \text{num_patches}, d_{\text{model}})$. To perform classification, this sequence of vectors must be aggregated into a single vector that represents the entire time-series. There are several common strategies for this aggregation:

1. **** Token Aggregation:**** Inspired by BERT, a special learnable vector, the (classification) token, is prepended to the sequence of patch embeddings before they enter the encoder. Because the self-attention mechanism allows every token to interact with every other token, the final hidden state corresponding to this token can be seen as an aggregate representation of the entire sequence, conditioned on all patches. This single vector of dimension d_{model} is then passed to the classification head.

2. **Global Average Pooling:** Another simple yet effective method is to take the mean of all the output patch embeddings across the sequence dimension. This results in a single vector of dimension d_{model} that summarizes the entire sequence.

The resulting aggregate vector is then fed into a standard classification head, which is typically a simple linear layer (`nn.Linear(d_model, num_classes)`) followed by a softmax function to produce the final class probabilities.

5.4 End-to-End PyTorch Tutorial for Hand Grip Classification

This section provides a complete, runnable PyTorch example for classifying simulated multivariate time-series data representing hand grip strategies.

5.4.1 Step 1: Setup and Data Simulation

First, we import the necessary libraries and create a synthetic dataset. We will simulate two classes of hand grips: a "power" grip with high-amplitude, low-frequency signals, and a "precision" grip with lower-amplitude, higher-frequency signals.

Python

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import numpy as np

# --- Data Simulation ---
def generate_grip_data(num_samples, seq_len, num_channels, grip_type):
    t = np.linspace(0, 10, seq_len)
    data = np.zeros((num_samples, seq_len, num_channels))
```

```

    for i in range(num_samples):
        for j in range(num_channels):
            noise = np.random.normal(0, 0.1, seq_len)
            if grip_type == 'power': # High amplitude, low frequency
                amplitude = np.random.uniform(1.5, 2.5)
                frequency = np.random.uniform(1, 2)
                data[i, :, j] = amplitude * np.sin(frequency * t) + noise
            elif grip_type == 'precision': # Low amplitude, high frequency
                amplitude = np.random.uniform(0.5, 1.0)
                frequency = np.random.uniform(4, 6)
                data[i, :, j] = amplitude * np.sin(frequency * t) + noise
        return data

# Generate data
n_samples = 1000
seq_length = 256
n_channels = 8

power_data = generate_grip_data(n_samples // 2, seq_length, n_channels, 'power')
precision_data = generate_grip_data(n_samples // 2, seq_length, n_channels,
'precision')

# Create labels (0 for power, 1 for precision)
power_labels = np.zeros(n_samples // 2)
precision_labels = np.ones(n_samples // 2)

# Combine and shuffle
X = np.concatenate((power_data, precision_data), axis=0)
y = np.concatenate((power_labels, precision_labels), axis=0)

indices = np.arange(n_samples)
np.random.shuffle(indices)
X = X[indices]
y = y[indices]

print(f"Data shape: {X.shape}") # (1000, 256, 8)
print(f"Labels shape: {y.shape}") # (1000,)

```

5.4.2 Step 2: Create a PyTorch Dataset

We create a custom Dataset to handle our time-series data. For simplicity, we will treat the entire sequence as input, but in a real-world scenario, this is where windowing or patching logic would be implemented.

Python

```
class GripDataset(Dataset):
    def __init__(self, data, labels):
        self.data = torch.tensor(data, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

# Split data and create DataLoaders
train_size = int(0.8 * len(X))
X_train, X_val = X[:train_size], X[train_size:]
y_train, y_val = y[:train_size], y[train_size:]

train_dataset = GripDataset(X_train, y_train)
val_dataset = GripDataset(X_val, y_val)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32)
```

5.4.3 Step 3: Define the Time-Series Transformer Model

Here we define our encoder-only Transformer for classification. It includes a linear

projection layer for the input patches (here, the features at each timestep), positional encoding, a standard Transformer Encoder, and a classification head.

Python

```
# Using the PositionalEncoding and EncoderLayer classes from Chapter 4
```

```
class TimeSeriesTransformer(nn.Module):
```

```
    def __init__(self, num_features, num_classes, d_model, nhead, num_layers, dim_feedforward, dropout=0.1):
```

```
        super().__init__()
```

```
        self.model_type = 'Transformer'
```

```
        self.d_model = d_model
```

```
        # Input embedding layer
```

```
        self.input_embedding = nn.Linear(num_features, d_model)
```

```
        # Positional Encoding
```

```
        self.pos_encoder = PositionalEncoding(d_model)
```

```
        # Transformer Encoder
```

```
        encoder_layers = nn.TransformerEncoderLayer(d_model, nhead, dim_feedforward, dropout, batch_first=True)
```

```
        self.transformer_encoder = nn.TransformerEncoder(encoder_layers, num_layers)
```

```
        # Classification Head
```

```
        self.classifier = nn.Linear(d_model, num_classes)
```

```
    def forward(self, src):
```

```
        # src shape: (batch_size, seq_length, num_features)
```

```
        # 1. Embed input
```

```
        src = self.input_embedding(src) * math.sqrt(self.d_model)
```

```
        # 2. Add positional encoding
```

```
        src = self.pos_encoder(src)
```

```

# 3. Pass through transformer encoder
output = self.transformer_encoder(src)
# output shape: (batch_size, seq_length, d_model)

# 4. Aggregate output for classification (use mean pooling)
output = output.mean(dim=1)

# 5. Pass through classification head
output = self.classifier(output)

return output

# Model parameters
num_features = n_channels
num_classes = 2
d_model = 64
nhead = 4
num_layers = 3
dim_feedforward = 128
dropout = 0.1

model = TimeSeriesTransformer(num_features, num_classes, d_model, nhead,
num_layers, dim_feedforward, dropout)

```

5.4.4 Step 4: Training and Evaluation Loop

Finally, we write the training and evaluation loop.

Python

```

# --- Training Setup ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
criterion = nn.CrossEntropyLoss()

```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
# --- Training Loop ---
```

```
num_epochs = 20
```

```
for epoch in range(num_epochs):
```

```
    model.train()
```

```
    total_loss = 0
```

```
    for data, labels in train_loader:
```

```
        data, labels = data.to(device), labels.to(device)
```

```
        optimizer.zero_grad()
```

```
        output = model(data)
```

```
        loss = criterion(output, labels)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
    total_loss += loss.item()
```

```
avg_train_loss = total_loss / len(train_loader)
```

```
# --- Validation Loop ---
```

```
model.eval()
```

```
total_val_loss = 0
```

```
correct = 0
```

```
total = 0
```

```
with torch.no_grad():
```

```
    for data, labels in val_loader:
```

```
        data, labels = data.to(device), labels.to(device)
```

```
        output = model(data)
```

```
        loss = criterion(output, labels)
```

```
        total_val_loss += loss.item()
```

```
        _, predicted = torch.max(output.data, 1)
```

```
        total += labels.size(0)
```

```
        correct += (predicted == labels).sum().item()
```

```
avg_val_loss = total_val_loss / len(val_loader)
```

```
accuracy = 100 * correct / total
```

```
print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_loss:.4f}, Val Loss: {avg_val_loss:.4f}, Accuracy: {accuracy:.2f}%")
```

This end-to-end example provides a concrete template for applying a Transformer to a multivariate time-series classification task. While this uses a simple mean pooling for aggregation, a more advanced implementation could incorporate the `` token strategy. For a more robust and feature-complete workflow, especially for larger datasets, leveraging the Hugging Face Trainer API is highly recommended. This involves wrapping the custom dataset in the datasets library format and using the Trainer class to handle the training loop, evaluation, and logging, which simplifies the process considerably.²⁰

Part IV: The Evolving Landscape of Transformer Models

Chapter 6: A Taxonomy of Modern Transformer Architectures

Since the publication of "Attention Is All You Need," the original encoder-decoder architecture has served as a foundational blueprint from which a diverse ecosystem of specialized models has evolved. These new architectures are not merely incremental improvements; they represent distinct philosophical approaches tailored to specific tasks. Understanding this taxonomy is crucial for situating one's own work and selecting the most appropriate tool for a given problem. The landscape can be broadly categorized into three dominant families: encoder-only models, decoder-only models, and Vision Transformers.

6.1 Encoder-Only Architectures (The "BERT" Family)

Encoder-only architectures, famously exemplified by BERT (Bidirectional Encoder Representations from Transformers), are designed primarily for Natural Language

Understanding (NLU) tasks. These tasks, such as sentiment analysis, named entity recognition, and sentence classification, require the model to develop a deep, contextual understanding of the entire input sequence.

- **Philosophy and Mechanism:** The core idea is to use the Transformer's encoder stack to generate a rich representation of each token that is conditioned on both its left and right contexts simultaneously—hence the term "bidirectional". This is achieved by feeding the entire input sequence into the model at once and allowing the self-attention mechanism in each layer to freely attend to all other tokens.
- **Pre-training Objectives:** To learn these powerful bidirectional representations from vast amounts of unlabeled text, BERT introduced two novel self-supervised pre-training tasks ¹⁸:
 1. **Masked Language Modeling (MLM):** In this task, approximately 15% of the input tokens are randomly masked (replaced with a special `` token), and the model's objective is to predict the original identity of these masked tokens based on the unmasked context. This forces the model to learn a deep understanding of language structure and context.
 2. **Next Sentence Prediction (NSP):** The model is given two sentences, A and B, and is trained to predict whether sentence B is the actual sentence that follows A in the original text or just a random sentence from the corpus. This task was intended to teach the model to understand relationships between sentences, which is important for downstream tasks like question answering. (It is worth noting that subsequent research, such as in the RoBERTa model, found the NSP objective to be less beneficial than initially thought and in some cases, detrimental to performance).
- **Relevance to Time-Series Classification:** The encoder-only paradigm is highly relevant for sequence classification tasks, including the classification of hand grip strategies. The objective is to *understand* the entire input time-series to produce a single classification label, which aligns perfectly with the design philosophy of models like BERT.

6.2 Decoder-Only Architectures (The "GPT" Family)

In contrast to the understanding-focused encoder models, decoder-only architectures like the Generative Pre-trained Transformer (GPT) family are designed for Natural Language Generation (NLG).²¹ Their primary function is to generate

coherent and contextually relevant text.

- **Philosophy and Mechanism:** These models are **autoregressive**, meaning they generate an output sequence one token at a time, with each new token being conditioned on all the previously generated tokens. They consist solely of a Transformer decoder stack. The key architectural feature that enables this behavior is the use of a **causal** or **look-ahead mask** within the self-attention mechanism. This mask prevents any position from attending to subsequent positions, ensuring that the prediction for token i can only depend on the known outputs at positions $1, \dots, i-1$.
- **Pre-training Objective:** The pre-training task for decoder-only models is straightforward next-token prediction. Given a sequence of text, the model is trained to predict the next token at every position in the sequence. This simple yet powerful objective, when applied at a massive scale, enables these models to learn grammar, facts, and even reasoning abilities from the training data.
- **Relevance to Time-Series Classification:** For a direct classification task, this architecture is less suitable than an encoder-only model. However, it is the go-to architecture for generative time-series tasks, such as forecasting or synthesizing new, realistic hand grip patterns based on an initial prompt.

6.3 Vision Transformers (ViT)

The Vision Transformer (ViT) represents a groundbreaking application of the Transformer architecture to the domain of computer vision, challenging the long-held dominance of Convolutional Neural Networks (CNNs).

- **Philosophy and Mechanism:** The core innovation of ViT is to treat an image not as a grid of pixels, but as a sequence of fixed-size patches.²² The process is as follows:
 1. **Image Patching:** An input image is divided into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels).
 2. **Embedding:** Each patch is flattened and linearly projected into a vector of the model's dimension (d_{model}).
 3. **Positional Encoding:** Positional embeddings are added to these patch embeddings to retain spatial information.
 4. **Encoder Processing:** This sequence of patch embeddings is then fed into a standard Transformer encoder. For classification, a special `` token is often prepended to the sequence, and its corresponding output from the encoder

is used as the final image representation for the classification head.

- Relevance to Time-Series Classification:** The ViT paradigm opens up a novel and powerful avenue for time-series analysis through the "time-series as an image" approach.²³ A multivariate time-series with C channels and T timesteps can be converted into a 2D image of size (C, T) . This image can then be processed by a ViT. This method allows the model to leverage the powerful, pre-trained backbones from the vision domain and provides a different way of capturing spatio-temporal patterns (where "spatial" now refers to the cross-channel dimension). This is a promising research direction for tasks like hand grip classification, where interactions between different sensor channels are crucial.

The following table provides a high-level summary of these three major architectural families, clarifying their core components, primary tasks, and key innovations.

Architecture Family	Core Component	Primary Task	Attention Type	Key Innovation	Example Models
Encoder-Only	Transformer Encoder	Understanding (NLU), Classification	Bidirectional Self-Attention	Masked Language Model (MLM) Pre-training	BERT, RoBERTa, DistilBERT
Decoder-Only	Transformer Decoder	Generation (NLG)	Causal (Masked) Self-Attention	Autoregressive Next-Token Prediction	GPT series, Llama, Mamba
Encoder-Decoder	Full Transformer	Sequence-to-Sequence	Bidirectional (Encoder), Causal (Decoder), Cross-Attention	Original architecture for translation	T5, BART, Original Transformer
Vision Transformer	Transformer Encoder	Image Classification / Vision Tasks	Bidirectional Self-Attention	Treating images as a sequence of patches	ViT, Swin Transformer, DeiT

Table 6.1: A comparative overview of the major Transformer architecture families. Each family leverages different components of the original architecture to excel at specific categories of tasks.

This taxonomy provides a clear framework for understanding the modern Transformer landscape. For the dissertation task of classifying hand grip strategies, the encoder-only architecture is the most direct and logical starting point. However, an awareness of the other paradigms, particularly the ViT's "time-series as an image" approach, offers exciting possibilities for novel research and comparative analysis.

Chapter 7: Taming Complexity: A Survey of Efficient Transformers

A significant limitation of the original Transformer architecture is the computational and memory complexity of its self-attention mechanism, which scales quadratically with the input sequence length, n . The complexity is $O(n^2 \cdot d)$, where d is the model's hidden dimension. This quadratic scaling makes processing very long sequences—such as high-resolution images, long documents, or high-frequency time-series data—prohibitively expensive or even impossible on current hardware.

This bottleneck has spurred a vibrant area of research focused on developing "efficient Transformers" that approximate the full self-attention mechanism while achieving linear or near-linear complexity. For a dissertation involving potentially long time-series from sensor data, understanding these efficient architectures is not just an academic exercise but a practical necessity. This chapter surveys three of the most influential approaches: Longformer, Linformer, and Performer.

7.1 Fixed Sparsity Patterns: The Longformer

The Longformer model tackles the complexity issue by introducing a sparse attention pattern that combines local and global context, making it particularly well-suited for tasks where both types of information are important.²⁴

- **Mechanism:** Instead of having every token attend to every other token, the Longformer's attention mechanism is a composite of three patterns:
 1. **Sliding Window Attention:** Most tokens employ a local, sliding window attention. Each token attends to a fixed window of w neighboring tokens (e.g., $w/2$ tokens to the left and $w/2$ to the right). This captures local context and has a linear complexity of $O(n \cdot w)$.
 2. **Dilated Sliding Window:** To increase the receptive field without increasing

computation, some layers can use a dilated window, which has gaps between the attended tokens. This helps information propagate across the sequence more quickly.

3. **Global Attention:** A small number of pre-selected tokens are designated to have "global" attention. These global tokens attend to all other tokens in the sequence, and all other tokens in the sequence attend to them. This is crucial for tasks that require a summary representation, such as classification, where the `` token is often given global attention.
- **Complexity and Trade-offs:** The combination of these patterns reduces the overall complexity to $O(n)$, making it possible to process sequences of thousands of tokens. The trade-off is that the attention pattern is sparse and fixed, meaning some long-range dependencies between non-global tokens might be missed in a single layer. However, by stacking multiple layers, information can propagate across the entire sequence.
- **Relevance to Time-Series:** The Longformer's philosophy is highly applicable to time-series data. Most temporal dependencies are local (a data point is most related to its immediate neighbors), which is handled well by the sliding window. At the same time, certain key events (e.g., the start of a grip motion) might need to be related to the entire sequence, which can be handled by assigning them global attention.

7.2 Low-Rank Approximation: The Linformer

The Linformer is based on a key theoretical insight: the self-attention matrix is often low-rank, meaning it can be well-approximated by the product of two smaller matrices. It exploits this property to reduce complexity without relying on sparse patterns.

- **Mechanism:** The core idea of the Linformer is to project the Key (K) and Value (V) matrices, which have a sequence length dimension of n , down to a much smaller, fixed dimension k (where $k \ll n$) before the attention computation. This is done by introducing two linear projection matrices, E and F , of size $(k \times n)$. The attention is then computed as: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{Q(EK^T)}{d})FV$
By re-associating the matrix multiplications, the expensive $O(n^2)$ computation of QKT is avoided. The complexity of this operation becomes $O(n \cdot k \cdot d)$, which is linear with respect to the sequence length n .
- **Complexity and Trade-offs:** The Linformer achieves linear complexity in both

time and space. Its performance is on par with standard Transformers on many tasks, while being significantly more efficient. The main trade-off is that its effectiveness depends on the choice of the projected dimension k . A smaller k is more efficient but may result in a greater loss of information from the approximation.

- **Relevance to Time-Series:** This is a powerful and general-purpose efficiency improvement. If the temporal dependencies in a hand grip signal can indeed be captured by a low-rank structure, the Linformer offers a theoretically sound and practically effective way to model long sequences.

7.3 Kernelization: The Performer

The Performer introduces a different approach to linearizing attention, using a mathematical technique called kernelization to approximate the softmax attention mechanism without ever explicitly forming the quadratic attention matrix.

- **Mechanism:** The Performer's key innovation is the FAVOR+ (Fast Attention Via Orthogonal Random Features) algorithm.²⁵ It relies on the property that the softmax function can be expressed as the expected value of the product of two positive random feature maps applied to the query and key vectors. In essence, it finds functions $\phi(Q')$ and $\phi(K')$ such that their dot product approximates the softmax of the original QKT dot product. This allows the attention computation to be reordered: $\text{Attention} \approx \phi(Q') \times (\phi(K')^T \times V)$
By first computing the product of the transformed keys and the values, the need to instantiate the full $n \times n$ attention matrix is completely bypassed.
- **Complexity and Trade-offs:** The Performer achieves linear space and time complexity and comes with strong theoretical guarantees that its approximation is unbiased or nearly-unbiased. It does not rely on any structural priors like sparsity or low-rankness. It has been shown to be exceptionally fast, especially for very long sequences, while retaining a high percentage of the original Transformer's accuracy.
- **Relevance to Time-Series:** The Performer offers a robust and highly efficient alternative to standard attention. Its ability to accurately approximate full-rank attention makes it a compelling choice for complex time-series where dependencies may not fit a simple sparse or low-rank structure.

The following table summarizes the key characteristics of these influential efficient Transformer architectures.

Model	Core Idea	Complexity	Key Advantage	Potential Drawback
Vanilla Transformer	Full Self-Attention	$O(n^2)$	Captures all pairwise interactions	Prohibitively expensive for long sequences
Longformer	Sparse Attention (Sliding Window + Global)	$O(n)$	Efficiently models local and key global dependencies	Sparsity pattern is fixed and may miss some interactions
Linformer	Low-Rank Matrix Approximation	$O(n)$	Theoretically grounded, simple projection	Performance depends on the choice of rank k
Performer	Softmax Kernel Approximation (FAVOR+)	$O(n)$	Provably accurate, no structural priors needed	Random feature mapping can introduce variance

Table 7.1: A comparison of efficient Transformer architectures. Each model offers a different strategy to overcome the quadratic complexity of the vanilla Transformer, presenting a trade-off between computational efficiency, modeling assumptions, and implementation complexity.

For a dissertation focused on long time-series data, the choice of which efficient Transformer to use is a critical experimental variable. The Longformer is often a strong, intuitive starting point for time-series due to its explicit handling of local context. The Linformer and Performer offer more general, mathematically-driven solutions that may capture more complex dependency structures. Comparing the performance of these different efficiency mechanisms on the hand grip classification task could itself be a valuable research contribution.

Part V: Advanced Topics and Future Directions

Chapter 8: Advanced Techniques for Time-Series Transformers

Beyond the core architecture and its efficient variants, the field of Transformers is continually evolving. For a researcher aiming to make a novel contribution, it is essential to be aware of the advanced techniques being developed, particularly those that address the unique challenges of time-series data. This chapter explores two such frontiers: self-supervised pre-training, which aims to leverage vast unlabeled datasets, and advanced positional encoding methods, which seek to better capture the nature of temporal information.

8.1 Self-Supervised Pre-training for Time-Series

One of the biggest challenges in applying deep learning to specialized domains like biomechanics is the scarcity of large, high-quality labeled datasets. Self-Supervised Learning (SSL) offers a powerful solution to this problem. The core idea is to first pre-train a model on a large corpus of *unlabeled* data using a "pretext task"—a task where the labels can be generated automatically from the data itself. This pre-training phase allows the model to learn rich, general-purpose representations of the data's underlying structure. The pre-trained model can then be fine-tuned on a much smaller labeled dataset for a specific downstream task, like classification, often achieving significantly better performance than a model trained from scratch.²⁶

For time-series data, several pretext tasks have been proposed:

- **Masked "Patch" Modeling:** This is a direct analogue to BERT's Masked Language Modeling (MLM) and is a natural fit for the patching approach described in Chapter 5.¹⁸ In this task, a certain percentage of the input time-series patches are randomly masked (e.g., replaced with zeros or a learned mask token). The model's objective is to predict the original values of these masked patches based on the context provided by the unmasked patches. To succeed, the model must learn the inherent temporal dynamics and correlations within the time-series data.¹⁸
- **Contrastive Learning:** This family of methods works by creating multiple "views"

of the same data sample through data augmentation. For a given time-series, one might create two augmented views by applying transformations like adding random noise, jittering (small time shifts), or scaling the amplitude. The model is then trained with a contrastive loss function. The objective is to pull the vector representations of "positive pairs" (different views of the *same* sample) closer together in the embedding space, while simultaneously pushing apart the representations of "negative pairs" (views from *different* samples). This encourages the model to learn representations that are invariant to noise and minor transformations but are highly specific to the core identity of the time-series.

- **Time-Frequency Consistency (TF-C):** This is a sophisticated and powerful contrastive learning strategy specifically designed for time-series.²⁷ It leverages the fundamental principle from signal processing that a signal can be equivalently represented in the time domain and the frequency domain (via a Fourier Transform). In the TF-C framework, the two "views" for the contrastive task are the time-domain representation and the frequency-domain representation of the same signal. The model is trained to produce consistent embeddings for these two views. This forces the model to learn representations that capture both the temporal evolution and the periodic characteristics of the signal, leading to a more robust and generalizable model.

For a dissertation on hand grip classification, the potential impact of SSL is immense. If a large dataset of unlabeled biomechanical sensor data is available (even if it's not specifically from hand grips), pre-training a Transformer using a task like masked patch modeling could allow the model to learn the fundamental "language" of physiological signals. This pre-trained model would then start the fine-tuning process with a much better initialization, likely leading to higher accuracy, faster convergence, and better generalization, especially when the labeled dataset for the specific grip strategies is small.

8.2 Advanced Positional Encodings for Time-Series

The standard sinusoidal positional encoding was designed for sequences of discrete, uniformly spaced tokens. Time-series data, however, often has different characteristics; it can be continuous, irregularly sampled, and the notion of relative position is often more important than absolute position. This has led to the

development of more advanced positional encoding schemes.

- **Learnable Positional Encodings:** Instead of using a fixed sinusoidal function, this approach treats the positional embeddings as learnable parameters of the model, typically implemented as a standard `nn.Embedding` layer where the input is the position index. This gives the model the flexibility to learn the optimal positional representation for the specific patterns in the training data. The primary drawback is that it cannot generalize to sequence lengths longer than the maximum length seen during training, and it adds to the total number of model parameters.
- **Relative Positional Encodings:** This approach argues that for many tasks, the absolute position of a token is less important than its relative distance to other tokens. Instead of adding a position vector to the input embeddings, relative positional encodings modify the attention calculation itself. They introduce a learnable bias that is dependent on the pairwise distance between the query and key tokens. This makes the attention scores sensitive to relative position, which is often a more natural way to model sequential dependencies.
- **Rotary Positional Encoding (RoPE):** RoPE is a novel and elegant implementation of relative positional encoding that has become a standard in many modern large language models. It encodes positional information by applying a rotation matrix to the Query and Key vectors, where the angle of rotation depends on the token's absolute position. A key property is that the dot product between two rotated vectors depends only on their original embeddings and their *relative* position. This method effectively injects relative position information directly into the self-attention mechanism, has excellent extrapolation properties to longer sequences, and has demonstrated strong performance.
- **Timestamp-based Encodings (e.g., tAPE):** For time-series data that is irregularly sampled, using a simple integer index as the position can be misleading. A more powerful approach is to directly encode the actual timestamp of each observation. For example, the **Time Absolute Position Encoding (tAPE)** method adapts the sinusoidal formula to incorporate the actual timestamp values, providing the model with true temporal context rather than just ordinal information. This is particularly relevant for medical or real-world sensor data where the time between measurements can vary significantly.

For the hand grip classification task, experimenting with these advanced positional encodings presents another clear avenue for research. Comparing the performance of the standard sinusoidal encoding against learnable embeddings and the more sophisticated RoPE could yield valuable insights into which form of positional

information is most beneficial for modeling biomechanical signals.

Chapter 9: Interpretability and Model Diagnostics

While achieving high accuracy is a primary goal in machine learning, for scientific applications such as a dissertation, understanding *why* a model makes its predictions is equally, if not more, important. Interpretability transforms a model from a "black box" into a scientific tool that can be used to validate hypotheses and generate new insights. For Transformer models, the attention mechanism itself provides a powerful, built-in tool for diagnostics and interpretation.

9.1 The Promise of Attention Maps

The core of the self-attention mechanism is the computation of attention weights. For a sequence of length n , these weights form an $n \times n$ matrix for each attention head in each layer. This matrix, often called an "attention map," can be visualized as a heatmap. The value at position (i,j) in this map represents the strength of the attention that token i (the query) pays to token j (the key) when computing its updated representation.

By visualizing these attention maps, one can gain insight into the model's internal reasoning process. It allows us to ask questions like: Which parts of the input sequence did the model focus on most when making its decision? Are there consistent patterns in what the model attends to across different examples of the same class? Do these patterns align with existing domain knowledge?

9.2 Visualizing Attention for Time-Series Classification

In the context of classifying hand grip strategies, visualizing attention maps can be particularly insightful. For an encoder-only model, the attention maps from the final layers are often the most informative. By plotting the time-series signal and overlaying

a heatmap derived from the attention weights, one can directly see which segments of the grip motion the model deemed most important for its classification decision.

For example, a visualization might reveal that for "power grip" classifications, the model consistently places high attention on the initial, high-amplitude phase of the signal. Conversely, for "precision grip" classifications, it might focus on later segments characterized by fine, high-frequency adjustments. Such a finding would not only increase confidence in the model but could also provide quantitative evidence supporting existing biomechanical theories.

9.2.1 Tutorial: Visualizing Attention with PyTorch and BertViz

Several excellent tools exist for visualizing attention. BertViz is a popular, interactive library designed for Hugging Face models, but the principles can be applied to any PyTorch Transformer.²⁸ Here is a conceptual guide to visualizing attention maps for a custom time-series Transformer.

1. **Extract Attention Weights:** The first step is to modify the model to output the attention weights during the forward pass. For Hugging Face models, this is as simple as setting `output_attentions=True` during model instantiation. For a custom PyTorch model, one would modify the forward method of the `MultiHeadAttention` class to return the `attention_probs` tensor alongside the final output.

```
Python
# In a custom MultiHeadAttention class's forward method
#...
attn_probs = torch.softmax(scores, dim=-1)
output = torch.matmul(attn_probs, V)
return output, attn_probs # Return weights along with the output
```

2. **Run Inference and Collect Weights:** Pass a sample time-series through the model in evaluation mode and collect the attention weights from the desired layer and head. The attention weights will typically have a shape of `(batch_size, num_heads, seq_length, seq_length)`.
3. **Process and Plot:** Select a specific sample, layer, and head to visualize. The corresponding `(seq_length, seq_length)` matrix can be plotted as a heatmap using libraries like `matplotlib` or `seaborn`.

```
Python
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
def plot_attention_map(attention_weights, input_data, title="Attention Map"):
```

```
    """
```

```
    Visualizes an attention map for a single time-series sample.
```

```
    Args:
```

```
        attention_weights (torch.Tensor): Attention weights of shape (seq_len, seq_len).
```

```
        input_data (torch.Tensor): The original time-series data of shape (seq_len, num_features).
```

```
        title (str): Title for the plot.
```

```
    """
```

```
    fig, axs = plt.subplots(2, 1, figsize=(12, 8), gridspec_kw={'height_ratios': })
```

```
    # Plot the time-series data (e.g., the first channel)
```

```
    axs.plot(input_data[:, 0].cpu().numpy())
```

```
    axs.set_title("Input Time-Series (Channel 0)")
```

```
    axs.set_xlabel("Time Step")
```

```
    axs.set_ylabel("Sensor Value")
```

```
    # Plot the attention heatmap
```

```
    sns.heatmap(attention_weights.cpu().numpy(), ax=axs, cmap='viridis')
```

```
    axs.set_title(title)
```

```
    axs.set_xlabel("Key (Attended To)")
```

```
    axs.set_ylabel("Query (Attending From)")
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
# --- Inside an evaluation loop ---
```

```
# model.eval()
```

```
# output, all_attention_weights = model(sample_data) # Assuming model returns weights
```

```
# Select weights for the first sample, last layer, first head
```

```
# last_layer_attention = all_attention_weights[-1] # List of weights per layer
```

```
# attention_to_plot = last_layer_attention[0, 0, :, :].detach() # (batch=0, head=0)
```

```
# plot_attention_map(attention_to_plot, sample_data)
```

This code provides a basic template for creating visualizations. Libraries like BertViz offer more interactive features, such as line-based visualizations that connect attending tokens, which can be even more intuitive.

The ability to perform this kind of model diagnostic is a significant advantage of the Transformer architecture. For a scientific dissertation, it elevates the analysis from simply reporting accuracy scores to generating deeper, mechanistic insights. By correlating attention patterns with known physiological events in the hand grip data, the model can be used as a computational tool to confirm, refine, or even challenge existing knowledge in the field of biomechanics. This bridge between predictive performance and scientific interpretability is where some of the most impactful research can be done.

Chapter 10: Conclusion and Dissertation Recommendations

This report has provided a comprehensive exploration of the Transformer architecture and its underlying attention mechanism, journeying from the foundational principles that motivated its creation to the advanced techniques that define the current state of the art. We have deconstructed the original "Attention Is All You Need" model, translated its components into practical PyTorch code, and detailed the specific adaptations required for applying these powerful models to the domain of time-series classification, with a particular focus on the analysis of hand grip strategies.

10.1 Summary of Key Learnings

The analysis has revealed several critical points. First, the Transformer's success is rooted in its ability to overcome the fundamental limitations of recurrent models, namely the sequential computation bottleneck and the difficulty of modeling long-range dependencies. The self-attention mechanism, by creating direct, content-based connections between all elements in a sequence, provides a highly parallelizable and effective solution to these challenges. Second, the architecture itself is a masterclass in synergistic design, where components like positional encoding, multi-head attention, residual connections, and layer normalization each play a crucial and complementary role. Third, the generic nature of the Transformer allows it to be adapted to a wide variety of data types beyond text, including time-series, through clever "tokenization" strategies like patching. Finally, the field is rapidly evolving, with ongoing research into efficiency improvements, advanced

positional encodings, and self-supervised pre-training methods offering fertile ground for novel contributions.

10.2 Proposed Architecture for Hand Grip Classification

Based on this comprehensive analysis, a robust and promising architectural starting point for a dissertation on classifying hand grip strategies would be an **encoder-only Transformer**. This architecture is philosophically aligned with the classification task, which requires a deep understanding of the input sequence rather than sequence generation.

Specific recommendations for the architecture include:

- **Input Processing:** The multivariate time-series data should be processed using a **patching or windowing** approach. A channel-mixing strategy, where the features from all sensor channels within a patch are flattened and linearly projected into a single embedding, is recommended to allow the model to learn cross-channel correlations early on. The patch size should be treated as a key hyperparameter for experimentation.
- **Attention Mechanism:** Given that biomechanical time-series can be long, especially with high sampling rates, utilizing an **efficient Transformer** is advisable. The **Longformer** architecture is an excellent starting point, as its combination of a local sliding window and global attention is intuitively well-suited to time-series data, which often exhibits both local continuity and key global events.
- **Positional Encoding:** While standard sinusoidal encodings provide a solid baseline, experimenting with **learnable positional encodings** is recommended for their ability to adapt to the specific dataset. For more advanced exploration, implementing and evaluating **Rotary Positional Encoding (RoPE)** could represent a significant technical contribution.
- **Classification Head:** The **token aggregation** strategy is a robust and widely-used method. A special ```` token should be prepended to the sequence of patch embeddings, and its final hidden state from the encoder should be passed to a simple linear layer for the final classification.

10.3 Avenues for Novel Contribution

Building upon this recommended architecture, several avenues for novel research present themselves, which could elevate the dissertation from a simple application of existing methods to a meaningful scientific contribution:

1. **Self-Supervised Pre-training:** This is arguably the most impactful area for a novel contribution. The researcher could collect a large corpus of unlabeled biomechanical time-series data (e.g., from public datasets like PhysioNet or other available sensor data). Pre-training the proposed Transformer architecture on this large dataset using a pretext task like **masked patch modeling** could lead to a powerful, domain-specific foundation model. Demonstrating that this pre-trained model significantly outperforms a model trained from scratch on the smaller, labeled hand grip dataset would be a major finding.
2. **Architectural Comparison:** A rigorous comparison between different approaches to time-series "tokenization" could be a core part of the dissertation. This would involve comparing the performance of the proposed 1D patching Transformer against a **Vision Transformer (ViT)-based approach**, where the multivariate time-series is converted into a 2D image. Analyzing the performance trade-offs and the different patterns learned by each architecture would provide valuable insights.
3. **Interpretability as a Scientific Tool:** Beyond reporting classification metrics, a deep dive into the model's interpretability could yield novel biomechanical insights. By systematically **visualizing the attention maps** for different grip types, the research could identify and quantify which parts of the temporal signal (e.g., onset, peak force, release) and which sensor channels the model deems most critical for distinguishing between strategies. This could be used to validate or even challenge existing theories in motor control and biomechanics.

10.4 Final Words

The Transformer architecture has fundamentally reshaped the landscape of deep learning. Its application to domains beyond natural language, such as the analysis of complex biological signals, is still a burgeoning field ripe with opportunity. By combining a solid theoretical understanding with rigorous practical implementation and a focus on scientific inquiry, the framework outlined in this report provides a clear

path toward a successful and impactful dissertation. The ability of these models to learn intricate, long-range dependencies from sequential data holds immense promise for unlocking new levels of understanding in fields from genomics to neuroscience and biomechanics. The journey ahead is complex but filled with the potential for exciting discovery.

Works cited

1. What is the different between using standard RNN and LSTM for time series prediction?, accessed on July 12, 2025, <https://community.deeplearning.ai/t/what-is-the-different-between-using-standard-rnn-and-lstm-for-time-series-prediction/202995>
2. Recurrent Neural Networks (RNNs): Challenges and Limitations | by Indrajitbarat | Medium, accessed on July 12, 2025, <https://medium.com/@indrajitbarat9/recurrent-neural-networks-rnns-challenges-and-limitations-4534b25a394c>
3. Advantages and Limitations of Recurrent Neural Networks (RNNs) in Sequence Modeling, accessed on July 12, 2025, <https://www.kaggle.com/general/446729>
4. Limitations of RNNs & LSTMs for Sequences - ApX Machine Learning, accessed on July 12, 2025, <https://apxml.com/courses/foundations-transformers-architecture/chapter-1-revisiting-sequence-modeling-limitations>
5. The Limitations of Recurrent Neural Networks (RNNs) and Why They Matter - Medium, accessed on July 12, 2025, <https://medium.com/@yonasdesta2012/the-limitations-of-recurrent-neural-networks-rnns-and-why-they-matter-eb0a05c90b60>
6. Do RNNs/LSTMs really need to be sequential? - Artificial Intelligence Stack Exchange, accessed on July 12, 2025, <https://ai.stackexchange.com/questions/27171/do-rnns-lstms-really-need-to-be-sequential>
7. The Attention Mechanism from Scratch - MachineLearningMastery.com, accessed on July 12, 2025, <https://machinelearningmastery.com/the-attention-mechanism-from-scratch/>
8. Attention Mechanism in Deep Learning - Analytics Vidhya, accessed on July 12, 2025, <https://www.analyticsvidhya.com/blog/2019/11/comprehensive-guide-attention-mechanism-deep-learning/>
9. Attention is All you Need - NIPS, accessed on July 12, 2025, <https://papers.neurips.cc/paper/7181-attention-is-all-you-need.pdf>
10. What is an attention mechanism? | IBM, accessed on July 12, 2025, <https://www.ibm.com/think/topics/attention-mechanism>
11. In Depth Understanding of Attention Mechanism (Part II) - Scaled ..., accessed on July 12, 2025, <https://medium.com/@funcry/in-depth-understanding-of-attention-mechanism->

[part-ii-scaled-dot-product-attention-and-its-7743804e610e](#)

12. A Technical Overview of the Attention Mechanism in Deep Learning | by Silva.f.francis, accessed on July 12, 2025, <https://medium.com/@silva.f.francis/a-technical-overview-of-the-attention-mechanism-in-deep-learning-a6a11264226a>
13. Multi-Head Attention Mechanism - GeeksforGeeks, accessed on July 12, 2025, <https://www.geeksforgeeks.org/nlp/multi-head-attention-mechanism/>
14. A Gentle Introduction to Positional Encoding in Transformer Models, Part 1 - MachineLearningMastery.com, accessed on July 12, 2025, <https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>
15. Layer Normalization Explained | Papers With Code, accessed on July 12, 2025, <https://paperswithcode.com/method/layer-normalization>
16. The Feedforward Network (FFN) in The Transformer Model | by ..., accessed on July 12, 2025, <https://medium.com/image-processing-with-python/the-feedforward-network-fn-in-the-transformer-model-6bb6e0ff18db>
17. Time Series Classification with Transformers – Center for Applied AI, accessed on July 12, 2025, <https://caai.ai.uky.edu/time-series-classification-with-transformers/>
18. liaoyuhua/pretrained-time-series-model: Pretrain a time ... - GitHub, accessed on July 12, 2025, <https://github.com/liaoyuhua/pretrained-time-series-model>
19. A Transformer-based Framework for Multivariate Time Series ..., accessed on July 12, 2025, <https://arxiv.org/abs/2010.02803>
20. Fine-tuning - Hugging Face, accessed on July 12, 2025, <https://huggingface.co/docs/transformers/training>
21. What is GPT (generative pre-trained transformer)? | IBM, accessed on July 12, 2025, <https://www.ibm.com/think/topics/gpt>
22. Vision Transformers (ViT) Explained | Pinecone, accessed on July 12, 2025, <https://www.pinecone.io/learn/series/image-search/vision-transformers/>
23. Vision Transformers for Time Series Forecasting - Lessons Learned, accessed on July 12, 2025, <https://www.dakshineshwari.net/post/vision-transformers-for-time-series-forecasting-lessons-learned>
24. Longformer - Hugging Face, accessed on July 12, 2025, https://huggingface.co/docs/transformers/model_doc/longformer
25. Rethinking Attention with Performers | by Nakshatra Singh | Oct ..., accessed on July 12, 2025, <https://medium.com/analytics-vidhya/paper-explained-rethinking-attention-with-performers-b207f4bf4bc5>
26. qingsongedu/Awesome-SSL4TS: A professionally curated ... - GitHub, accessed on July 12, 2025, <https://github.com/qingsongedu/Awesome-SSL4TS>
27. Self-Supervised Contrastive Pre-Training For Time Series - Zitnik Lab, accessed on July 12, 2025, <https://zitniklab.hms.harvard.edu/projects/TF-C/>
28. jessevig/bertviz: BertViz: Visualize Attention in NLP Models ... - GitHub, accessed on July 12, 2025, <https://github.com/jessevig/bertviz>